

HƯỚNG DẪN CUSTOM LOGIN VỚI SPRING BOOT 4 +SPRING SECURITY7+MAPPER+THYMELEAF

Bài tập: Cho bảng User, Role hãy viết chức năng custom login có thể đăng nhập bằng username, email đều được, thông tin fullname và images của user sẽ hiển thị ở header.html. Sử dụng spring boot 4 và spring security, mapstruct.

Bước 1: tạo project với file pom.xml sau:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <groupId>vn.iotstar</groupId>
  <artifactId>springboot1-9</artifactId>
  <version>1.0</version>
  <name>springboot1-9</name>
  <description/>
  <url/>
  <licenses>
    <license/>
  </licenses>
  <developers>
    <developer/>
  </developers>
  <scm>
    <connection/>
    <developerConnection/>
    <tag/>
    <url/>
  </scm>
  <properties>
    <java.version>26</java.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-h2console</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
```

```

        <artifactId>spring-boot-starter-mail</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>
    <dependency>
        <groupId>org.thymeleaf.extras</groupId>
        <artifactId>thymeleaf-extras-springsecurity6</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>com.microsoft.sqlserver</groupId>
        <artifactId>mssql-jdbc</artifactId>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-mail-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependency>

```

```

        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.mapstruct</groupId>
        <artifactId>mapstruct</artifactId>
        <version>1.6.3</version>
    </dependency>
    <dependency>
        <groupId>nz.net.ultraq.thymeleaf</groupId>
        <artifactId>thymeleaf-layout-dialect</artifactId>
        <scope>compile</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <executions>
                <execution>
                    <id>default-compile</id>
                    <phase>compile</phase>
                    <goals>
                        <goal>compile</goal>
                    </goals>
                    <configuration>
                        <annotationProcessorPaths>
                            <path>
                                <groupId>org.mapstruct</groupId>
                                <artifactId>mapstruct-processor</artifactId>
                                <version>1.6.3</version>
                            </path>
                        </annotationProcessorPaths>
                    </configuration>
                </execution>
            </executions>
        </plugin>
    </plugins>
</build>

```

```

        <groupId>org.projectlombok</groupId>
        <artifactId>lombok-mapstruct-binding</artifactId>
        <version>0.2.0</version>
    </path>
</path>

<groupId>org.projectlombok</groupId>

<artifactId>lombok</artifactId>

</path>
<path>

<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-
configuration-processor</artifactId>
</path>
</annotationProcessorPaths>
</configuration>
</execution>
<execution>
    <id>default-testCompile</id>
    <phase>test-compile</phase>
    <goals>
        <goal>testCompile</goal>
    </goals>
    <configuration>
        <annotationProcessorPaths>
            <path>

<groupId>org.projectlombok</groupId>

<artifactId>lombok</artifactId>

</path>
</annotationProcessorPaths>
</configuration>
</execution>
</executions>
</plugin>
</plugins>
</build>

</project>

```

Bước 2: Tạo entity

```

package vn.iotstar.entity;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "roles")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor

```

```

@Builder
public class Role {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String name;
}
}

package vn.iotstar.entity;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(
    name = "users",
    uniqueConstraints = {
        @UniqueConstraint(name = "uk_users_username",
            columnNames = "username"),
        @UniqueConstraint(name = "uk_users_email",
            columnNames = "email")
    }
)
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String username;

    @Column(nullable = false, unique = true, length = 150)
    private String email;

    @Column(nullable = false)
    private String password;

    @Column(name = "full_name", length = 150, columnDefinition = "nvarchar(200)")
    private String fullName;

    @Column(length = 500)
    private String images;

    @Column(nullable = false)
    @Builder.Default
    private boolean enabled = true;
}

```

```

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "role_id", nullable = false)
    private Role role;
}

```

Bước 3: tạo repository

```

package vn.iotstar.repository;

import vn.iotstar.entity.Role;

import java.util.Optional;

import org.springframework.data.jpa.repository.JpaRepository;

public interface RoleRepository extends JpaRepository<Role, Long> {

    Optional<Role> findByName(String name);
}

```

```

package vn.iotstar.repository;

import vn.iotstar.entity.User;

import java.util.Optional;

import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Long> {

    Optional<User> findByUsername(String username);

    Optional<User> findByEmail(String email);

    Optional<User> findByUsernameOrEmail(
        String username,
        String email
    );
}

```

Bước 4: tạo dto

```

package vn.iotstar.dto;

import lombok.*;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder

```

```

public class UserDTO {

    private Long id;

    private String username;

    private String email;

    private String fullName;

    private String images;

    private String roleName;

    private boolean enabled;
}

```

```

package vn.iotstar.dto;

import jakarta.validation.constraints.NotBlank;
import lombok.*;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class LoginDTO {

    @NotBlank(message = "Username hoặc email không được để trống")
    private String login;

    @NotBlank(message = "Password không được để trống")
    private String password;
}

```

Bước 5: tạo Mapper

```

package vn.iotstar.mapper;

import org.mapstruct.Mapper;
import org.mapstruct.Mapping;

import vn.iotstar.dto.UserDTO;
import vn.iotstar.entity.User;

@Mapper(componentModel = "spring")
public interface UserMapper {

    @Mapping(
        target = "roleName",
        source = "role.name"
    )
    UserDTO toDTO(User user);
}

```

Bước 6: tạo security

```

package vn.iotstar.security;

import java.util.Collection;
import java.util.List;

import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;

import lombok.Getter;

@Getter
public class CustomUserDetails implements UserDetails {

    private static final long serialVersionUID = 1L;

    private final Long id;

    private final String username;

    private final String email;

    private final String password;

    private final String fullName;

    private final String images;

    private final String role;

    private final boolean enabled;

    public CustomUserDetails(
        Long id,
        String username,
        String email,
        String password,
        String fullName,
        String images,
        String role,
        boolean enabled
    ) {
        this.id = id;
        this.username = username;
        this.email = email;
        this.password = password;
        this.fullName = fullName;
        this.images = images;
        this.role = role;
        this.enabled = enabled;
    }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {

```

```

        return List.of(
            new SimpleGrantedAuthority(role)
        );
    }

    @Override
    public String getPassword() {
        return password;
    }

    @Override
    public String getUsername() {
        return username;
    }

    @Override
    public boolean isAccountNonExpired() {
        return true;
    }

    @Override
    public boolean isAccountNonLocked() {
        return true;
    }

    @Override
    public boolean isCredentialsNonExpired() {
        return true;
    }

    @Override
    public boolean isEnabled() {
        return enabled;
    }
}

```

```

package vn.iotstar.security;

```

```

import lombok.RequiredArgsConstructor;

```

```

import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

```

```

import vn.iotstar.entity.User;
import vn.iotstar.repository.UserRepository;

```

```

@Service
@RequiredArgsConstructor
public class CustomUserDetailsService
    implements UserDetailsService {

    private final UserRepository userRepository;

```

```

@Override
public UserDetails loadUserByUsername(String login)
    throws UsernameNotFoundException {

    User user = userRepository
        .findByUsernameOrEmail(login, login)
        .orElseThrow(() ->
            new UsernameNotFoundException(
                "Không tìm thấy username/email: " + login
            )
        );

    return new CustomUserDetails(
        user.getId(),
        user.getUsername(),
        user.getEmail(),
        user.getPassword(),
        user.getFullName(),
        user.getImages(),
        user.getRole().getName(),
        user.isEnabled()
    );
}
}

```

Bước 7: tạo config security

```

package vn.iotstar.config;

import lombok.RequiredArgsConstructor;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;

import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;

import org.springframework.security.config.annotation.web.builders.HttpSecurity;

import org.springframework.security.core.userdetails.UserDetailsService;

import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableMethodSecurity
@RequiredArgsConstructor

```

```

public class SecurityConfig {

    private final UserDetailsService userDetailsService;

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public DaoAuthenticationProvider authenticationProvider() {

        DaoAuthenticationProvider provider =
            new DaoAuthenticationProvider(userDetailsService);

        provider.setPasswordEncoder(passwordEncoder());

        return provider;
    }

    @Bean
    public AuthenticationManager authenticationManager(
        AuthenticationConfiguration configuration
    ) throws Exception {

        return configuration.getAuthenticationManager();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(
        HttpSecurity http
    ) throws Exception {

        http
            .authorizeHttpRequests(auth -> auth

                .requestMatchers(
                    "/login",
                    "/css/**",
                    "/js/**",
                    "/images/**",
                    "/uploads/**"
                ).permitAll()

                .requestMatchers("/admin/**")
                .hasRole("ADMIN")

                .anyRequest()
                .authenticated()
            )

            .formLogin(form -> form

                .loginPage("/login")
            )
        }
    }

```

```

        .loginProcessingUrl("/login")

        .defaultSuccessUrl(
            "/",
            true
        )

        .failureUrl(
            "/login?error=true"
        )

        .permitAll()
    )

    .logout(logout -> logout

        .logoutUrl("/logout")

        .logoutSuccessUrl("/login?logout=true")

        .invalidateHttpSession(true)

        .deleteCookies("JSESSIONID")

        .permitAll()
    );

    return http.build();
}
}

```

Bước 8: tạo controller

```

package vn.iotstar.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class AuthController {

    @GetMapping("/login")
    public String login() {
        return "auth/login";
    }
}

```

```

package vn.iotstar.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class HomeController {

    @GetMapping("/")

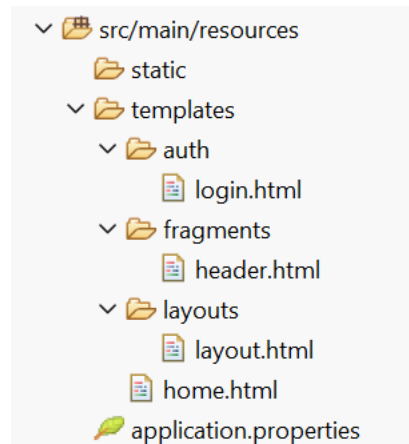
```

```

    public String home() {
        return "home";
    }
}

```

Bước 9: Tạo views



Login.html

```

<!DOCTYPE html>
<html lang="vi"
      xmlns:th="http://www.thymeleaf.org">

<head>

    <meta charset="UTF-8">

    <title>Đăng nhập</title>

    <meta name="viewport"
          content="width=device-width, initial-scale=1">

</head>

<body>

<h2>Đăng nhập</h2>

<div th:if="${param.error}">
    <p style="color:red">
        Username/email hoặc password không đúng
    </p>
</div>

<div th:if="${param.logout}">
    <p style="color:green">
        Bạn đã đăng xuất
    </p>
</div>

<form th:action="@{/Login}"

```

```
        method="post">

<div>

    <label>
        Username hoặc Email
    </label>

    <input
        type="text"
        name="username"
        placeholder="Username hoặc email"
        required>

</div>

<div>

    <label>
        Password
    </label>

    <input
        type="password"
        name="password"
        placeholder="Password"
        required>

</div>

<button type="submit">
    Đăng nhập
</button>

</form>

</body>
</html>
```

Layout.html

```
<!DOCTYPE html>

<html lang="vi"
    xmlns:th="http://www.thymeleaf.org"
    xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout">

<head>

    <meta charset="UTF-8">

    <title layout:title-pattern="$CONTENT_TITLE - UTEShop">
        UTEShop
    </title>
```

```

</head>

<body>

<header th:replace=~{fragments/header :: header}">
</header>

<main layout:fragment="content">

</main>

<footer>

    Copyright © UTEShop

</footer>

</body>

</html>

```

Header.html

```

<!DOCTYPE html>
<html lang="vi"
      xmlns:th="http://www.thymeleaf.org">

<body>

<header th:fragment="header">

    <nav>

        <a th:href="@{/}">
            UTEShop
        </a>

        <div th:if="${#authorization.expression('isAuthenticated()')}">

            <!-- Avatar -->


            <!-- Full name -->

            <span
                th:text="${#authentication.principal.fullName}">

```

```
        Nguyễn Văn A
    </span>

    <!-- Username -->

    <span
        th:text="'(' + ${#authentication.principal.username} + ')'">
        (nguyenvana)
    </span>

    <!-- Email -->

    <span
        th:text="${#authentication.principal.email}">
        email@gmail.com
    </span>

    <!-- Role -->

    <span
        th:text="${#authentication.principal.role}">
        ROLE_USER
    </span>

    <!-- Logout -->

    <form
        th:action="@{/Logout}"
        method="post"
        style="display:inline">

        <button type="submit">
            Đăng xuất
        </button>

    </form>

</div>

<div th:unless="${#authorization.expression('isAuthenticated()')}">

    <a th:href="@{/Login}">
        Đăng nhập
    </a>

</div>

</nav>

</header>

</body>
</html>
```

Home.html

```

<!DOCTYPE html>

<html lang="vi"
      xmlns:th="http://www.thymeleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorate="~{layouts/layout}">

<head>

    <title>Trang chủ</title>

</head>

<body>

<main layout:fragment="content">

    <h1>
        Trang chủ UTEShop
    </h1>

    <p>
        Xin chào
        <strong
            th:text="${#authentication.principal.fullName}">
            User
        </strong>
    </p>

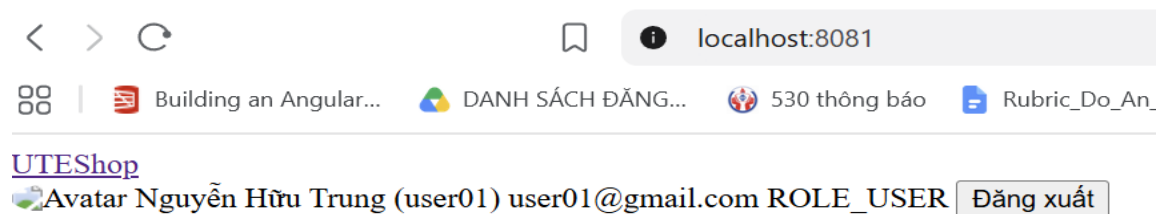
</main>

</body>

</html>

```

Bước 10: kết quả



Trang chủ UTEShop

Xin chào **Nguyễn Hữu Trung**

Copyright © UTEShop

Bước 11: tạo dữ liệu mẫu

```
package vn.iotstar;
```

```

import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import org.springframework.security.crypto.password.PasswordEncoder;

import vn.iotstar.entity.Role;
import vn.iotstar.entity.User;
import vn.iotstar.repository.RoleRepository;
import vn.iotstar.repository.UserRepository;

@SpringBootApplication
public class Springboot19Application {

    public static void main(String[] args) {
        SpringApplication.run(Springboot19Application.class, args);
    }

    @Bean
    CommandLineRunner init(
        RoleRepository roleRepository,
        UserRepository userRepository,
        PasswordEncoder passwordEncoder
    ) {

        return args -> {

            Role userRole = roleRepository
                .findByName("ROLE_USER")
                .orElseGet(() -> {
                    roleRepository.save(
                        Role.builder()
                            .name("ROLE_USER")
                            .build()
                    );
                });

            if (userRepository
                .findByUsername("user01")
                .isEmpty()) {

                User user = User.builder()
                    .username("user01")
                    .email("user01@gmail.com")
                    .password(
                        passwordEncoder.encode("123456")
                    )
                    .fullName("Nguyễn Hữu Trung")
                    .images("/images/user.png")
                    .role(userRole)
                    .enabled(true)
                    .build();

                userRepository.save(user);
            }
        }
    }
}

```

```
};
```

```
}
```

```
}
```